

Last updated on **Apr 29, 2026**

Write queries and explore data in the new SQL editor

The Databricks UI includes a SQL editor that you can use to author queries, collaborate with colleagues, browse available data, and create visualizations. This page explains how to use the SQL editor to write, run, manage, and share queries.

This article explains how to use the new SQL editor. To learn about working with the legacy SQL editor, see [Write queries and explore data in the legacy SQL editor](#).

NOTE

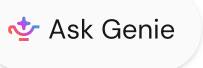
Starting in late May 2026, the new SQL editor will be enabled by default for all workspaces. The ability to turn off the new SQL editor at the workspace level will no longer be available. Individual users will still be able to switch to the legacy editor during this period. Starting in late July 2026, the legacy SQL editor will be retired and all users will use the new SQL editor. See [What's coming?](#) for details.

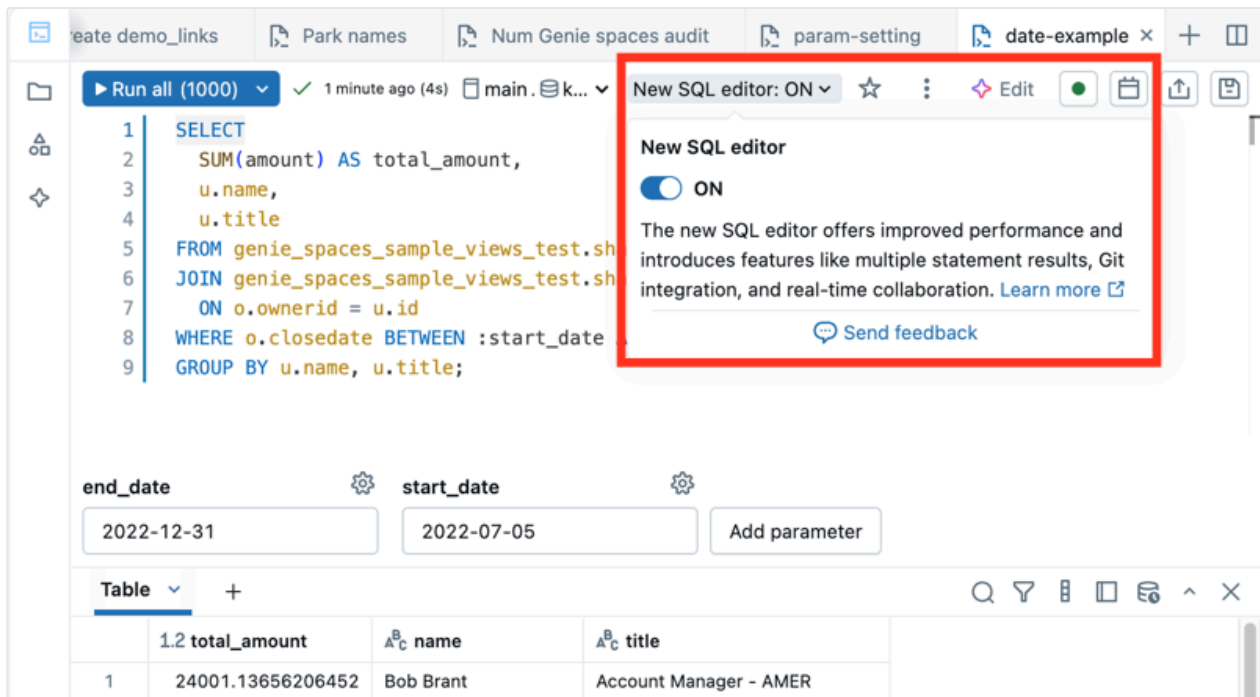
Open the SQL editor

To open the SQL editor in the Databricks UI, click  **SQL Editor** in the sidebar.

The SQL editor opens to your last open query. If no query exists, or all of your queries have been explicitly closed, the SQL editor landing page opens.

Turn on the new SQL editor

Use the **New SQL editor** toggle, to the right of the catalog and schema drop-down selectors, to turn the new editor on. On narrow displays, you can find the **New SQL editor** toggle by clicking the kebab menu  to the right of the catalog a  Ask Genie selectors.



Enable the new editor for any query that you own:

- Click **New SQL editor: OFF**. A **New SQL editor** popover menu opens.
 - The **Apply to all of my queries** checkbox is selected by default. Keep this setting to enable the editor for all queries that you own.
 - To enable the new editor for the active query only, unselect the **Apply to all of my queries** checkbox.
- Set the **New SQL editor** toggle to **ON**.

The new SQL editor cannot be used for all queries. You cannot switch to the new SQL editor for a query if any of the following are true:

- You do not own the query.
- The query contains **Query-based Dropdown Lists**, which are not supported. See [Use named parameter markers](#).

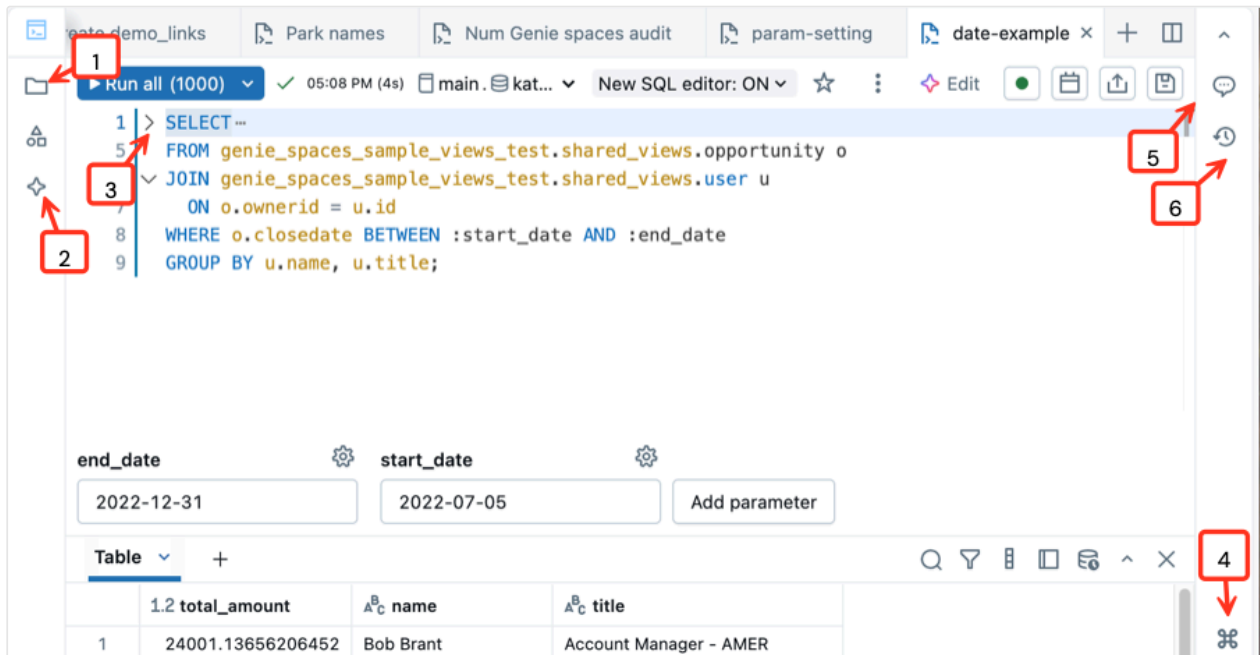
When you turn on the new SQL editor, your query reopens in the new UI. The result set is empty until you run the query.

NOTE

Query results are shared with all collaborators and are limited to 64,000 rows.

Orientation

This section highlights key differences in the UI and explains how to find and use available features.

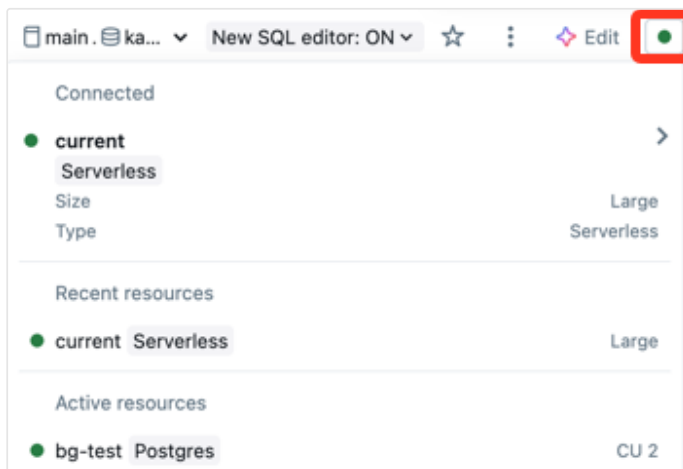


- 1. Access the workspace file system:** You can access and organize your workspace objects directly from the SQL editor.
- 2. Use the integrated Genie Code:** Chat with Genie Code to help you write, refine, or understand queries. Run code generated by Genie Code from the side panel.
- 3. Collapse sections of code:** Click the collapse arrow to the left of a code block to selectively hide and expand code sections. This can make working through large files easier.
- 4. Open the command palette:** The command palette includes common actions, keyboard shortcuts, custom themes, and links to help you quickly find what you need. After you open it, use the search bar to look for specific tools and shortcuts.
- 5. Comment on code:** Add comments to code to enhance collaboration.
- 6. Review version history:** Review recent changes to your query.

Connect to compute

You must have at least CAN USE permissions on a SQL warehouse to run queries. You can use the drop-down near the top of the editor to see available options. To filter the

list, enter text in the text box.



If you have a default SQL warehouse, the SQL editor automatically uses it when you create a query. If no default warehouse is set, you select from an alphabetical list of available warehouses. Subsequent queries use the last selected warehouse. To set a default warehouse, see [Set a user-level default warehouse](#).

The icon next to the SQL warehouse indicates the status:

- Running 
- Stopped 

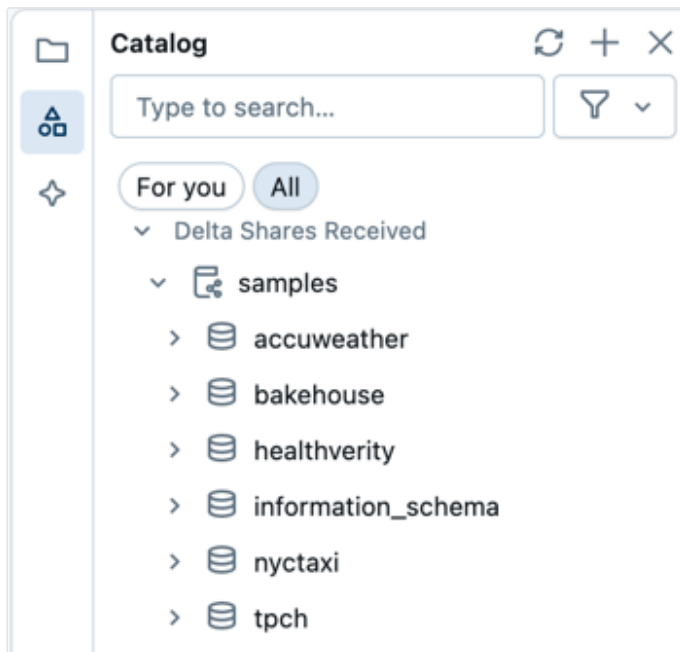
NOTE

If there are no SQL warehouses in the list, contact your workspace administrator.

The selected SQL warehouse will restart automatically when you run your query. See [Start a SQL warehouse](#) to learn other ways to start a SQL warehouse.

Browse assets and get help

Use the left pane in the SQL editor to browse workspace files, view data objects, and get help from Databricks Assistant.



Browse workspace files

Click  the folder icon to open your workspace user folder. You can navigate across all of the workspace files you have access to from this part of the UI.

Browse data objects

If you have metadata read permission, the schema browser in the SQL editor shows the available databases and tables. You can also browse data objects from [Catalog Explorer](#).

You can navigate Unity Catalog-governed database objects in Catalog Explorer without active compute. To explore data in the `hive_metastore` and other catalogs not governed by Unity Catalog, you must attach to compute with appropriate privileges. See [Data governance with Databricks](#).

NOTE

If no data objects exist in the schema browser or Catalog Explorer, contact your workspace administrator.

Click  near the top of the schema browser to refresh the schema. You can type in the search bar to filter assets by name. Click the  filter icon to filter objects by type.

Click the name of an object in the browser to see more details on the object. For example, click a schema name to show the tables in that schema. Click a table name to show the columns in that table.

Get help from Genie Code

Click  Genie Code icon to open a chat window with Genie Code. Click a suggested question or enter your own question to interact with Genie Code.

Create a query

You can enter text to create a query in the SQL editor. You can insert elements from the schema browser to reference catalogs and tables.

1. Type your query in the SQL editor.

The SQL editor supports autocomplete. As you type, autocomplete suggests completions. For example, if a valid completion at the cursor location is a column, autocomplete suggests a column name. If you type `select * from table_name as t where t.`, autocomplete recognizes that `t` is an alias for `table_name` and suggests the columns inside `table_name`. You can also use autocomplete to reference [query snippets](#).

```

1 SELECT
2   *
3 FROM
4   sales_customers sc
5 where
6   sc.

```

	address	string
	city	string
	continent	string
	country	string
	customerID	bigint
	email_address	string
	first_name	string
	gender	string
	last_name	string
	phone_number	string
	postal_zip_code	bigint
	state	string

2. (Optional) When you are done editing, click **Save**. By default, the query is saved to your user home folder. To save the query to a different location, select the target folder and click **Move**.

NOTE

New queries are automatically named **New query** with the creation timestamp appended in the title. By default, new queries created without a specific folder context are created in the **Drafts** folder in your home directory. When new queries are saved or renamed, they are removed from **Drafts**.

Query data sources

You can identify a query source using a [fully-qualified table name](#) in the query itself or by selecting a combination of catalog and schema from the drop-down selectors along with the table name in the query. A fully-qualified table name in the query overrides the catalog and schema selectors in the SQL editor. If a table or column name includes spaces, wrap those identifiers in backticks in your SQL queries.

NOTE

 Ask Genie

The maximum number of results returned in a table is 64,000 rows or 10MB, whichever is smaller.

The following examples demonstrate how to query various table-like objects that you can store in a catalog.

Query a standard table or view

The following example queries a table from the `samples` catalog.

SQL

```
SELECT
  o_orderdate,
  o_orderkey,
  o_custkey,
  o_totalprice,
  o_shippriority
FROM
  samples.tpch.orders
```

Query a metric view

The following example queries a metric view that uses a table from the `samples` catalog as its source. It evaluates the three listed measures and aggregates over `Order Month` and `Order Status`. It returns results sorted by `Order Month`. To create a similar metric view in your workspace, see [Tutorial: Build a complete metric view with joins](#).

All measure evaluations must be wrapped in the `MEASURE` function. See [measure aggregate function](#).

SQL

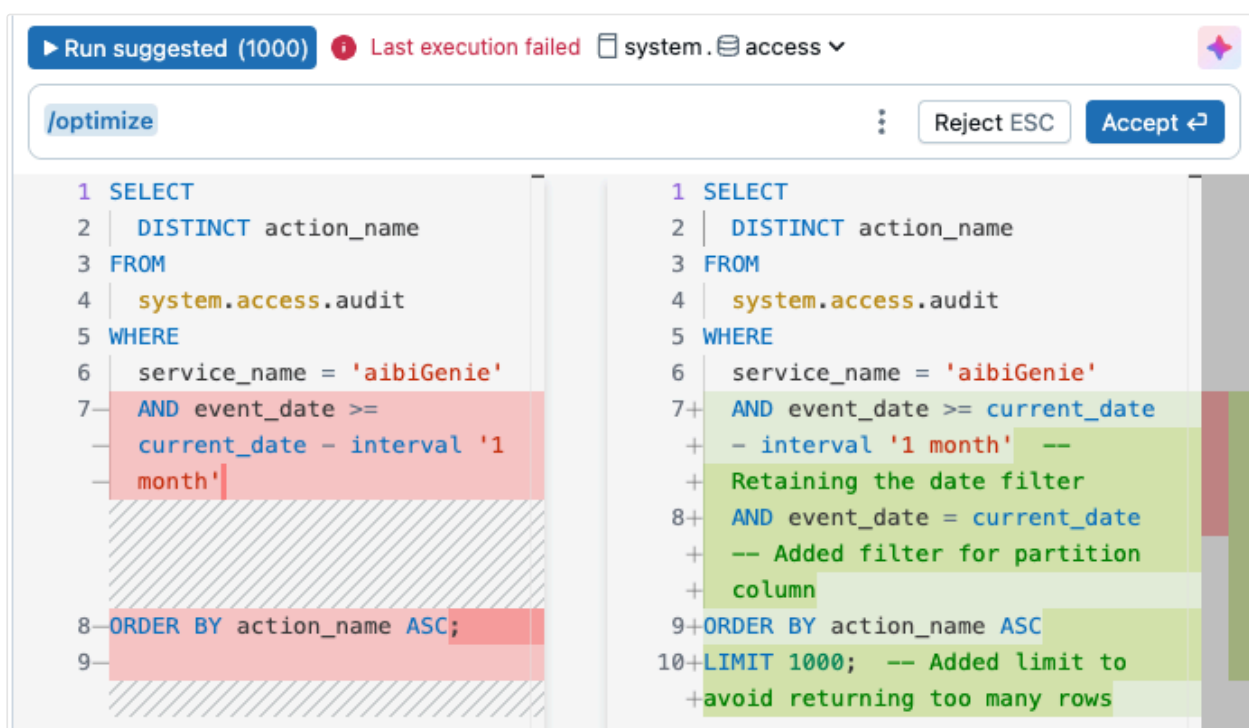
```
SELECT
  `Order Month`,
  `Order Status`,
  MEASURE(`Order Count`),
  MEASURE(`Total Revenue`),
  MEASURE(`Total Revenue per Customer`)
FROM
```

 Ask Genie

```
orders_metric_view
GROUP BY ALL
ORDER BY 1 ASC;
```

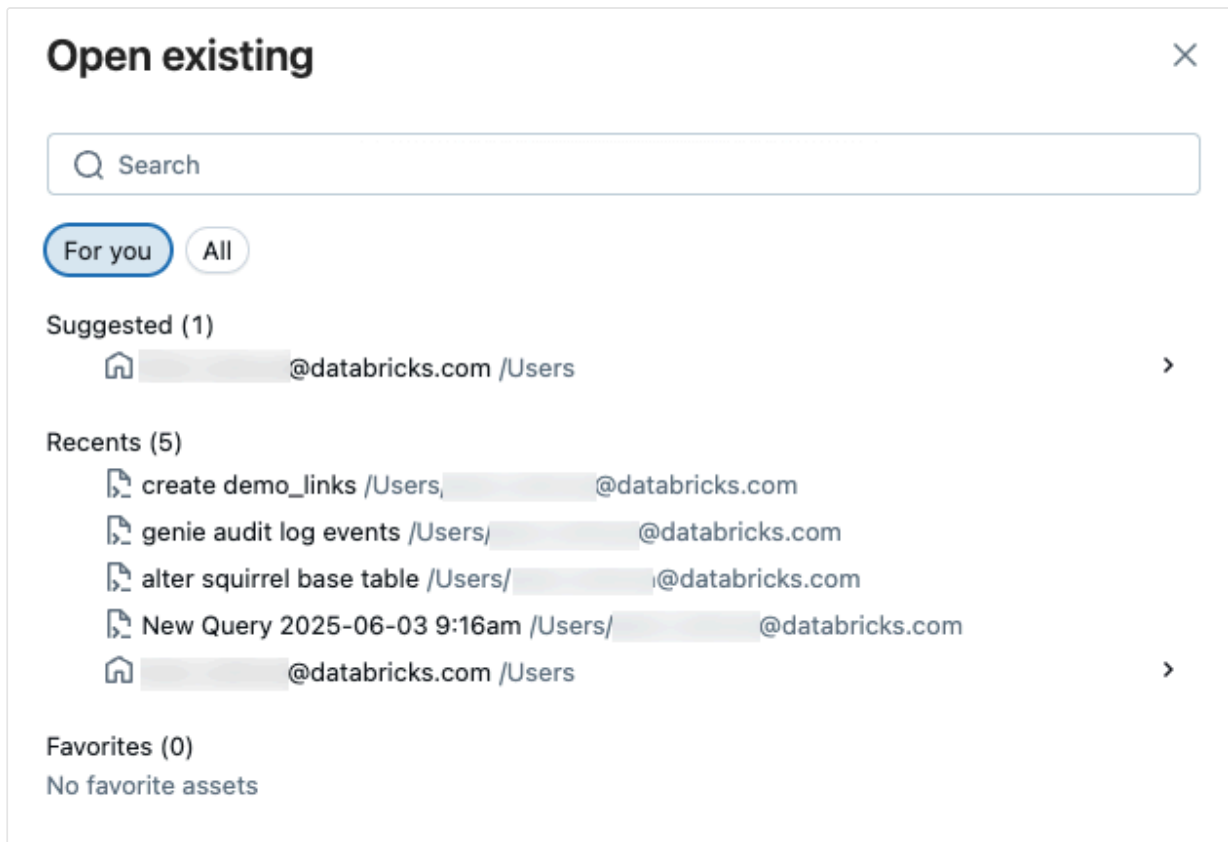
Optimize a query with Genie Code

Click the  Assistant icon on the right side of the editor to get inline help and suggestions when writing queries. The `/optimize` slash command prompts Assistant to evaluate and optimize queries. For more information, see [Optimize Python, PySpark, and SQL code](#).



Edit multiple query tabs

By default, the SQL editor uses tabs so you can open and edit multiple queries simultaneously. To open a new tab, click **+**, then select **Create new query** or **Open existing query**. Click **Open existing query** to see a list of queries. The **For you** tab offers a curated list of suggestions based on your usage. Use the **All** tab to find any query that you have access to.



Run a single query or multi-statement queries

You can write multi-statement queries into a single editor tab. To use multiple query statements in a single tab, end each statement with a semicolon (;).

To run a single query:

1. Select a SQL warehouse.
2. Highlight a query in the SQL editor (if multiple queries are in the query pane).
3. Press **Ctrl/Cmd + Enter** or click **Run (1000)** to display the results as a table in the results pane.

NOTE

By default, **Run all statements** and **Limit 1000** are selected.

- To run only the active statement, unselect **Run all statements**.
- If a query is saved with the **Limit 1000** setting, this setting applies to all query runs. To return all rows (up to 64,000 rows or 10 MB, whichever is

 Ask Genie

smaller), clear **LIMIT 1000** from the **Run (1000)** drop-down menu. To specify a different limit on the number of rows, add a `LIMIT` clause in your query.

Terminate a query

To terminate a query while it is running, click **Cancel**. An administrator can stop an running query that another user started by viewing the [Terminate an executing query](#).

Share a query

The new SQL editor handles permissions in two parts:

Interactive execution: Queries in the new SQL editor always run using **Run as viewer** credentials. Query results are accessible to anyone who has access to the query. Users with CAN EDIT permissions or higher can modify the shared query draft. Additionally, unsaved edits are visible to anyone with CAN VIEW permissions or above. Users must have at least CAN USE permission on the associated warehouse to run queries.

Credential settings: The permission settings for legacy alerts and [Workflows](#) are the same as those for the legacy SQL editor. Depending on your preferences, these assets can be run using either the owner's or the viewer's credentials.

See [Configure query permissions](#) to learn about **Run as owner** and **Run as viewer** credential settings. See [Query ACLs](#) to learn how to manage access to the query object.

Collaborate on queries in real time

```
1 SELECT
2   T.pickup_zip,
3   T.dropoff_zip,
4   T.route as `Route`,
5   T.frequency as `Number Trips`,
6   T.total_fare as `Total Revenue`
7 FROM
8   (
9     SELECT
10      pickup_zip,
11      dropoff_zip,
12      concat(pickup_zip, '-', dropoff_zip) AS route,
13      count(*),
14      SUM(fare_amount) as total_fare
15    FROM
16
17    GROUP BY
18      1,2,3
19  ) T
20 ORDER BY
21   1 ASC
```

After sharing the query, you can coauthor or debug queries live through collaborative editing. Editors can also comment on individual lines of code to ask questions or add suggestions.

Source control a query

Databricks SQL query files (extension: `.dbquery.ipynb`) are supported in [Databricks Git folders](#). You can use a Git folder to source control your query files and to share them in other workspaces with Git folders that access the same Git repository. If you choose to opt out of the new SQL editor after committing or cloning a query in a Databricks Git folder, delete and reclone that Git folder to avoid unexpected behaviors.

Copy the legacy query ID

To get the legacy query ID for a query that has been migrated to the new SQL editor, copy it from the **File** menu. This can be useful for API calls or other integrations that reference the legacy query ID.

1. Open the query in the new SQL editor.

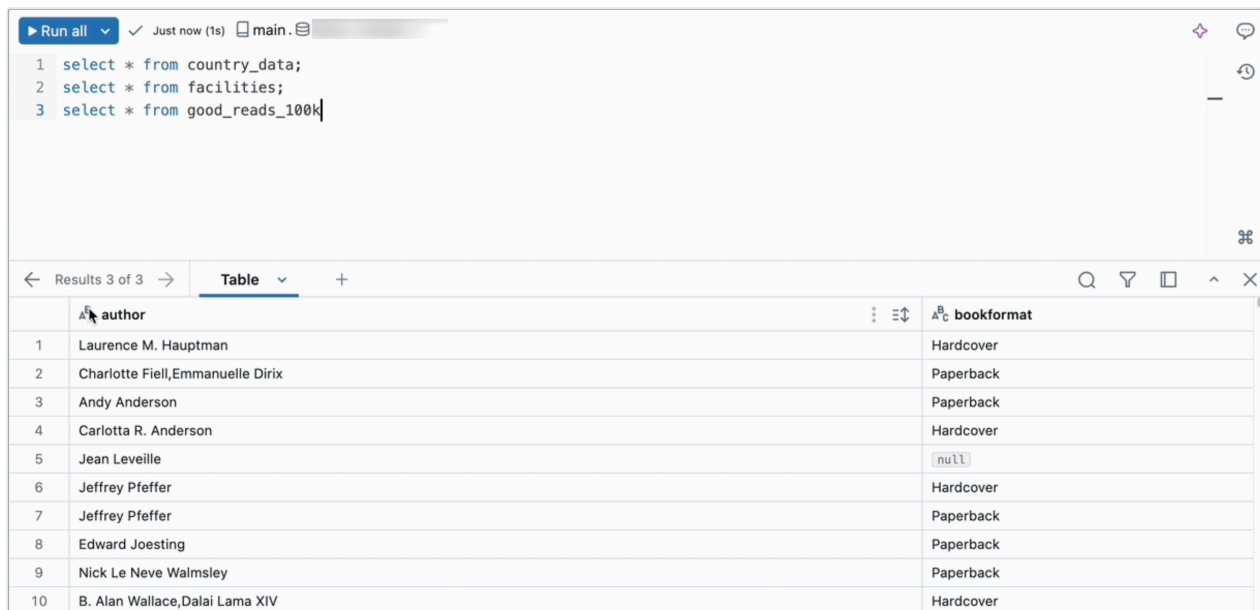
2. Click the kebab menu  at the top of the editor.
3. Click **File** > **Copy legacy query ID**.

Save a query

Query content in the new SQL editor is continuously autosaved. The **Save** button controls whether the draft query content should be applied to related assets, like workflows or legacy alerts. If the query is shared with the **Run as owner** credential, only the query owner can use the **Save** button to propagate changes. If the credential is set to **Run as viewer**, any user with at least `CAN MANAGE` permission can save the query.

View query results

You can compare the output from each statement with multiple result statements. When you run multiple SQL statements from a single edit window, you can use the UI in the **Output** panel near the bottom of the screen to scroll through the different result sets.



The screenshot shows the SQL editor interface. At the top, there's a 'Run all' button and a status bar indicating 'Just now (1s)' and 'main'. Below this, the SQL query is displayed in a code editor:

```
1 select * from country_data;
2 select * from facilities;
3 select * from good_reads_100k
```

Below the query editor, the 'Results' panel is visible, showing 'Results 3 of 3'. The results are displayed in a table format with two columns: 'author' and 'bookformat'. The table contains 10 rows of data:

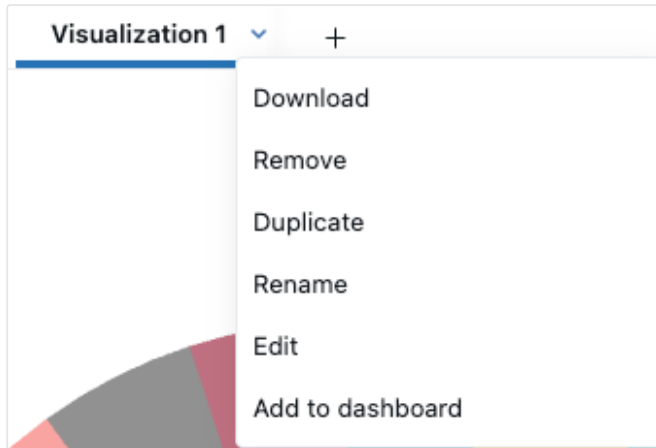
	author	bookformat
1	Laurence M. Hauptman	Hardcover
2	Charlotte Fiell,Emmanuelle Dirix	Paperback
3	Andy Anderson	Paperback
4	Carlotta R. Anderson	Hardcover
5	Jean Leveille	null
6	Jeffrey Pfeffer	Hardcover
7	Jeffrey Pfeffer	Paperback
8	Edward Joesting	Paperback
9	Nick Le Neve Walmsley	Paperback
10	B. Alan Wallace,Dalai Lama XIV	Hardcover

You can interact with and explore your query results using the result pane. The result pane includes the following features for exploring results.

Visualizations

Click the  to add a visualization. Visualizations can help explore the result set. See [Notebook and SQL editor visualization types](#) for a complete list of available visualization types.

Click the  on a visualization tab to show additional options.



- Click **Download** to download the data represented in the visualization as a CSV, TSV, or Excel file.
- Click **Remove** to delete the visualization.
- Click **Duplicate** to clone the results tab.
- Click **Rename** to retitle the visualization.

1. Click **Edit** to edit the visualization.

2. Click **Add to dashboard** to copy the query and visualization to a new [AI/BI dashboard](#).

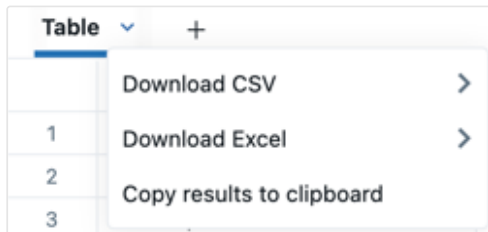
3. Select **Create new dashboard** or **Add to existing dashboard**.

- **Create new dashboard:** Creates a new dashboard that includes all the visualizations associated with the query. You are prompted to choose a name for the new dashboard. The new dashboard is saved to your home folder.
- **Add to existing dashboard:** Adds your query and associated visualizations to an existing dashboard. Use the drop-down menu to choose the dashboard that the dataset and visualization should be added to.

Download results

You can download results as a CSV or Excel file. CSV downloads support up to approximately 5GB of data. Click the  in a results tab to view download options. The

final file download size might be slightly more or less than 5GB, as the 5GB limit is applied to an earlier step than the final file download.



NOTE

If you cannot download a query, your workspace administrator has disabled download for your workspace.

Filters

Filters allow you to limit the result set after a query has run. You can apply filters to selectively show different subsets of the data. Use Genie Code to help you add a filter for results.

To add a filter:

1. Click  the filter icon.
2. Enter text to describe the kind of filter you want to apply. A named filter is applied to the result set.

To edit a filter:

1. Click the name of the filter.
2. Use the drop-down menus and text fields to adjust the filter settings.
3. (Optional) Select **Case sensitive** to make the filter case sensitive.
4. (Optional) Click **Add OR condition** to add an additional filter.

Add parameters

Parameters allow you to limit the result set by substituting values into a query at runtime. See [Use named parameter markers](#) to learn how to apply parameters.

Parameter syntax changes

Queries written in the legacy SQL editor that use double curly braces ({{}}) to mark parameters must be modified to run in the new SQL editor. The new SQL editor uses the same parameter syntax as other Databricks tools, like AI/BI dashboards, Genie Spaces, and notebooks.

Parameters in the new SQL editor are defined using named parameter syntax. Named parameters are parameters in a query that are prefixed with a colon, for example

:parameter_name.

See [Named parameter markers](#) for the SQL reference documentation. See [Named parameter syntax examples](#) for examples of converting parameter syntax from the classic SQL editor to named parameter markers.

Supported filter types

The following table shows filter types supported in the new SQL editor. The second column indicates whether the filter also works in the legacy SQL editor.

Filter type	Supported in legacy editor
Is equal to	Yes
Is not equal to	No
Is one of	Yes
Is not one of	No
Contains	Yes
Does not contain	No
Starts with	Yes
Does not start with	No

Filter type	Supported in legacy editor
Ends with	No
Does not end with	No
Is null	No
Is not null	No
Greater than	No
Greater than or equal to	No
Less than	No
Less than or equal to	No
Between	Yes
Before	No
After	No
Is true	Yes
Is false	Yes
OR conditions	No

Use keyboard shortcuts

The new SQL editor includes many built-in keyboard shortcuts to help you work quickly. The following table identifies commonly used keyboard shortcuts.

Function	macOS shortcut	Windows shortcut
Run a query	Cmd + Enter	Ctrl + Enter
Run a single highlighted statement	Cmd + Shift + Enter	Ctrl + Shift + Enter
Autocomplete	Ctrl + Space	Ctrl + Space
Increase font size	Option + + (plus key)	Alt + + (plus key)
Decrease font size	Option + - (minus key)	Alt + - (minus key)
Format SQL code	Cmd + Shift + F	Ctrl + Shift + F
View all shortcuts	Esc then H	Esc then H

Disable the new SQL editor

NOTE

Starting in late May 2026, the workspace-level opt-out for the new SQL editor will be removed. Individual users can still switch to the legacy editor until late July 2026, when the legacy SQL editor will be fully retired. See [What's coming?](#) for details.

To disable the new SQL editor:

- Click **New SQL editor: ON**. A **New SQL editor** popover menu opens.
- Set the **New SQL Editor** toggle to **OFF**.
- In the confirmation dialog, choose to disable the new editor for the active query only or for all eligible queries.
 - The **Disable for all eligible queries** is unselected. Keep this setting if you want to disable the new editor for only the active query.

- To disable the new SQL editor for all your queries, select the checkbox **Disable for all eligible queries**.

To prevent queries from automatically opening in the new SQL editor:

1. In the Databricks workspace, click your username in the top bar and select **Settings**.
2. In the left pane, under **User**, click the **Developer** tab.
3. Turn off **Create and open all eligible queries in the new SQL editor**.

Workspace admins can turn off this feature at the workspace level by visiting the preview portal and searching for **SQL editor**. See [Manage Databricks previews](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Custom format SQL statements

ⓘ PREVIEW

This feature is in [Public Preview](#).

This article explains how to customize SQL auto-formatting options in the Databricks UI.

Overview

SQL formatting improves the readability and maintainability of your queries. By standardizing indentation, keyword casing, and other stylistic preferences, a well-formatted query is easier to understand, debug, and share with your team.

In Databricks, you can configure formatting options using a `.dbsql-formatter-config.json` file saved in your workspace home directory. After saving the file, your SQL code is automatically formatted based on your preferences when you run or format code in the editor or a SQL notebook.

How to configure SQL formatting settings

To configure your personal SQL formatting options:

1. Click  **Workspace** in the sidebar.
2. Click **Create > File**.
3. Name the file `.dbsql-formatter-config.json`.

 Ask Genie

- 4. Add your desired settings as key-value pairs in JSON format. See [Formatter options](#)
- 5. To apply the latest formatting settings, refresh the page.

Example configuration file

The following example shows a valid `.dbsql-formatter-config.json` configuration.

JSON

```
{
  "printWidth": 80,
  "indentationStyle": "spaces",
  "indentationWidth": 4,
  "keywordCasing": "uppercase",
  "shouldExpandExpressions": true
}
```

Formatter options

The following table lists the supported configuration options.

Option	Config file name	Allowed values	Default	De
Print width	<code>printWidth</code>	Any integer >= 0	<code>100</code>	Se lin for
Indentation style	<code>indentationStyle</code>	<code>'spaces'</code> , <code>'tabs'</code>	<code>'spaces'</code>	Sp to tak inc in is i
 Ask Genie				

Option	Config file name	Allowed values	Default	De
Indentation width	<code>indentationWidth</code>	Integer from 0 to 99	2	Nu us in is s 's
Keyword casing	<code>keywordCasing</code>	'uppercase', 'lowercase', 'none'	'none'	Co for key lea un
Function name casing	<code>functionNameCasing</code>	'uppercase', 'lowercase', 'none'	'none'	Co for fun 'n ca
Comma placement	<code>commaPosition</code>	'beginning', 'end'	'end'	Co co pla
Line breaks between statements	<code>numNewLinesBetweenStatements</code>	Integer from 0 to 99	1	Nu line be sta
Line breaks between clauses	<code>numNewLinesBetweenClauses</code>	Integer from 0 to 99	0	Nu line be wit sta

Option	Config file name	Allowed values	Default	De
Expand boolean expressions	<code>shouldExpandExpressions</code>	<code>true</code> , <code>false</code>	<code>true</code>	Ex exp se
Expand CASE statements	<code>shouldExpandCaseStatements</code>	<code>true</code> , <code>false</code>	<code>true</code>	Ex cla sta ow
Expand IN lists	<code>shouldExpandInStatements</code>	<code>true</code> , <code>false</code>	<code>false</code>	Ex IN se
Expand BETWEEN conditions	<code>shouldExpandBetweenConditions</code>	<code>true</code> , <code>false</code>	<code>false</code>	Ex BE co se
Break JOIN conditions	<code>shouldBreakOnJoinSections</code>	<code>true</code> , <code>false</code>	<code>true</code>	Br co se

Is this page

Was this page helpful?

☐ Yes

☐ No

[Data warehousing](#)[SQL editor](#)[Use the legacy SQL editor](#)

Last updated on **Apr 29, 2026**

Write queries and explore data in the legacy SQL editor

ⓘ IMPORTANT

The legacy SQL editor will be retired in late July 2026. Starting in late May 2026, the new SQL editor will be enabled by default for all workspaces and the workspace-level opt-out will be removed. Individual users can still switch to the legacy editor until late July 2026, when the legacy editor will be fully retired. Databricks recommends transitioning to the [new SQL editor](#). See [What's coming?](#) for details.

The Databricks UI includes a SQL editor that you can use to author queries, browse available data, and create visualizations. You can also share your saved queries with other team members in the workspace. This article explains how to use the SQL editor to write, run, and manage queries. To learn how to enable and work with the new SQL editor, see [Write queries and explore data in the new SQL editor](#).

[Ask Genie](#)

The screenshot displays the Databricks SQL Editor. On the left, a sidebar contains navigation icons. The 'Catalog' pane shows a search bar and a list of databases: hive_metastore, ml, samples, and system. The main editor area features tabs for 'Customer Value', 'top ten accounts by country', and 'Order Revenue'. The 'Order Revenue' tab is active, showing a SQL query and its results. The query filters for orders from 1994-01-01 to 1994-01-31, grouped by date and priority, and ordered by total price. The results table shows 145 rows with columns: Date, Priority, and Total Price. The status bar at the bottom indicates '12 s 751 ms | 145 rows returned' and 'Refreshed a minute ago'.

After opening the editor, you can author a SQL query or browse the available data. The text editor supports autocomplete, autoformatting, and various other keyboard shortcuts.

You can open multiple queries using the query tabs at the top of the text editor. Each query tab has controls for running the query, marking the query as a favorite, and connecting to a SQL warehouse. You can also **Save**, **Schedule**, or **Share** queries.

Open the SQL editor

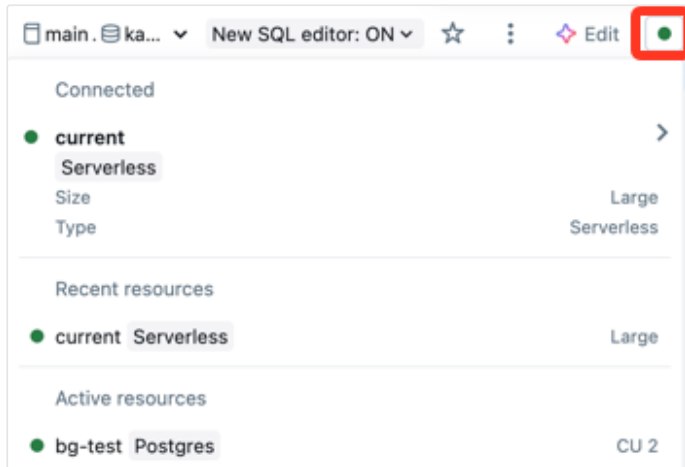
To open the SQL editor in the Databricks UI, click  **SQL Editor** in the sidebar.

The SQL editor opens to your last open query. If no query exists, or all of your queries have been explicitly closed, a new query opens. It is automatically named **New Query** and the creation timestamp is appended in the title.

If the **New SQL editor** is turned on, you can disable it to access the legacy SQL editor. See [Disable the new SQL editor](#).

Connect to compute

You must have at least CAN USE permissions on a running SQL warehouse to run queries. You can use the drop-down near the top of the editor to see available options. To filter the list, enter text in the text box.



The first time you create a query, the list of available SQL warehouses appears alphabetically. The last used SQL warehouse is selected the next time you create a query.

The icon next to the SQL warehouse indicates the status:

- Running
- Stopped

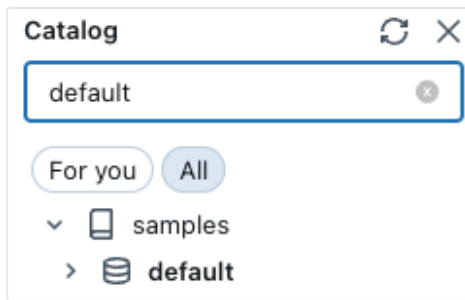
NOTE

If there are no SQL warehouses in the list, contact your workspace administrator.

The selected SQL warehouse will restart automatically when you run your query. See [Start a SQL warehouse](#) to learn other ways to start a SQL warehouse.

Browse data objects in SQL editor

If you have metadata read permission, the schema browser in the SQL editor shows the available databases and tables. You can also browse data objects from [Catalog Explorer](#).



You can navigate Unity Catalog-governed database objects in Catalog Explorer without active compute. To explore data in the `hive_metastore` and other catalogs not governed by Unity Catalog, you must attach to compute with appropriate privileges. See [Data governance with Databricks](#).

NOTE

If no data objects exist in the schema browser or Catalog Explorer, contact your workspace administrator.

Click  near the top of the schema browser to refresh the schema. You can filter the schema by typing filter strings in the search box.

Click a table name to show the columns for that table.

Create a query

You can enter text to create a query in the SQL editor. You can insert elements from the schema browser to reference catalogs and tables.

1. Type your query in the SQL editor.

The SQL editor supports autocomplete. As you type, autocomplete suggests completions. For example, if a valid completion at the cursor location is a column, autocomplete suggests a column name. If you type `select * from table_name as t where t.`, autocomplete recognizes that `t` is an alias for `table_name` and suggests the columns inside `table_name`.

```

1 SELECT
2   *
3 FROM
4   sales_customers sc
5 where
6   sc.

```

	address	string
	city	string
	continent	string
	country	string
	customerID	bigint
	email_address	string
	first_name	string
	gender	string
	last_name	string
	phone_number	string
	postal_zip_code	bigint
	state	string

2. (Optional) When you are done editing, click **Save**. By default, the query is saved to your user home folder, or you can select a different location. Then, click **Save**.

Query data sources

You can identify a query source using a [fully-qualified table name](#) in the query itself or by selecting a combination of catalog and schema from the drop-down selectors along with the table name in the query. A fully-qualified table name in the query overrides the catalog and schema selectors in the SQL editor. If a table or column name includes spaces, wrap those identifiers in backticks in your SQL queries.

The following examples demonstrate how to query various table-like objects that you can store in a catalog.

Query a standard table or view

The following example queries a table from the `samples` catalog.

SQL

```
SELECT
  o_orderdate,
  o_orderkey,
  o_custkey,
  o_totalprice,
  o_shippriority
FROM
  samples.tpch.orders
```

Query a metric view

The following example queries a metric view that uses a table from the samples catalog as its source. It evaluates the three listed measures and aggregates over `Order Month` and `Order Status`. It returns results sorted by `Order Month`. To create a similar metric view in your workspace, see [Tutorial: Build a complete metric view with joins](#).

All measure evaluations must be wrapped in the `MEASURE` function. See [measure aggregate function](#)

SQL

```
SELECT
  `Order Month`,
  `Order Status`,
  MEASURE(`Order Count`),
  MEASURE(`Total Revenue`),
  MEASURE(`Total Revenue per Customer`)
FROM
  orders_metric_view
GROUP BY ALL
ORDER BY 1 ASC;
```

Turn autocomplete on and off

Live autocomplete can complete schema tokens, query syntax identifiers (like `SELECT` and `JOIN`), and the titles of [query snippets](#). It's enabled by default unless your database schema exceeds five thousand tokens (tables or columns).

 Ask Genie

Use the toggle beneath the SQL editor to turn live autocomplete off or on.

- To turn off live autocomplete, press **Ctrl + Space** or click the  button beneath the SQL editor.

Optimize a query with Genie Code

The `/optimize` slash command prompts Assistant to evaluate and optimize queries. For more information, see [Optimize Python, PySpark, and SQL code](#).

Save queries

The **Save** button near the top-right of the SQL editor saves your query.

ⓘ IMPORTANT

When you modify a query but don't explicitly click **Save**, that state is retained as a query draft. Query drafts are retained for 30 days. After 30 days, query drafts are automatically deleted. To retain your changes, you must explicitly save them.

Edit multiple queries

By default, the SQL editor uses tabs so you can edit multiple queries simultaneously. To open a new tab, click **+**, then select **Create new query** or **Open existing query**. Click **Open existing query** to see your list of saved queries. click **My Queries** or **Favorites** to filter the list of queries. In the row containing the query you want to view, click **Open**.

Open query

My queries

Favorites

Trash

All queries

Q

Search queries

Name	Created at	
top ten accounts by country by	2023-12-07 15:34	<button>Focus</button>
New query by	2023-12-04 14:49	<button>Open</button>
Order Revenue quick-tag by	2023-10-12 16:08	<button>Focus</button>
National Revenue Map Sample by	2023-10-12 16:08	<button>Open</button>
TPCH - Number Suppliers Sample	2023-10-12 16:08	<button>Open</button>

Run a single query or multi-statement queries

To run a query statement or all query statements:

1. Select a SQL warehouse.
2. Highlight a query in the SQL editor (if multiple queries are in the query pane).
3. Press **Ctrl/Cmd + Enter** or click **Run (1000)** to display the results as a table in the results pane.

New query

Run (1000)

hive_meta.... def...

Serverless Starter W... (S)

Save*

Schedule

Share

```

1 SELECT year(birthDate) as birthYear, count(*) AS total
2 FROM default.people10m
3 WHERE firstName = 'Mary' AND gender = 'F'
4 GROUP BY birthYear
5 ORDER BY birthYear

```

Results

#	birthYear	total
1	1952	27
2	1953	25
3	1954	15
4	1955	23
5	1956	28
6	1957	29
7	1958	26
8	1959	22

1

2

>

19.28 s runtime | 49 rows

Refreshed 3 minutes ago

NOTE

By default, **Limit 1000** is selected to restrict query results to 1,000 rows.

- If the query is saved with **Limit 1000**, this setting applies to all query runs, including in dashboards.
- To return all rows (up to 64,000), deselect **Limit 1000** from the **Run (1000)** drop-down menu.
- To specify a different row limit, add a LIMIT clause to your SQL query.

The maximum number of rows returned in a table is 64,000.

Terminate a query

To terminate a query while it is running, click **Cancel**. An administrator can stop an running query that another user started by viewing the [Terminate an executing query](#).

Ask Genie

Query options

You can use the  kebab context menu near the top of the query editor to access menu options to clone, revert, format, and edit query information.

Revert to saved query

When you edit a query, a **Revert changes** option appears in the context menu for the query. You can click **Revert** to go back to your saved version.

Discarding and restoring queries

To move a query to trash:

- Click the kebab context menu  next to the query in the SQL editor and select **Move to Trash**.
- Click **Move to trash** to confirm.

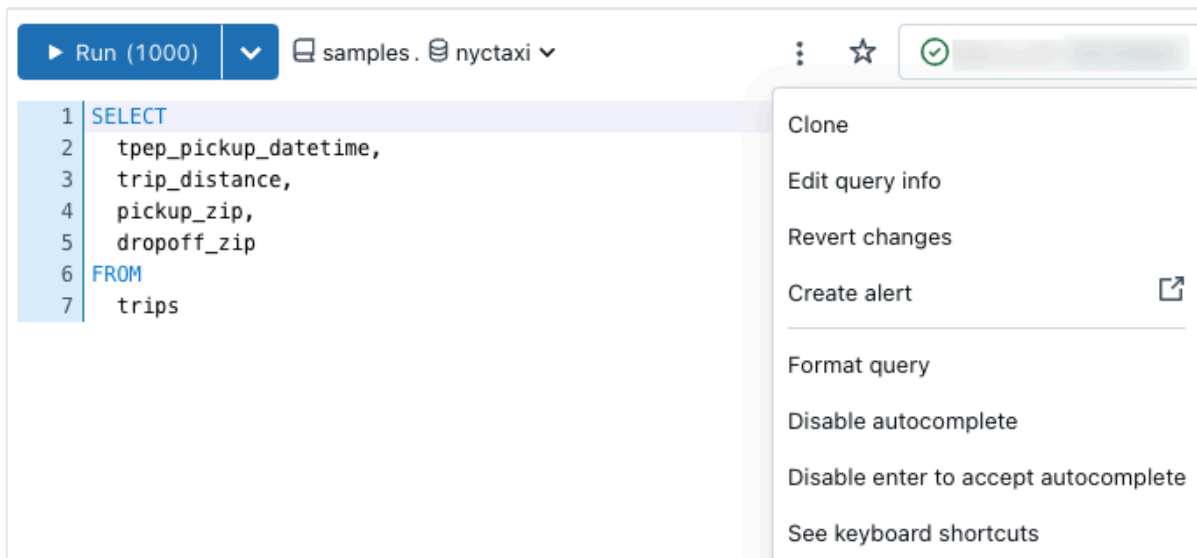
To restore a query from trash:

1. In the All Queries list, click .
2. Click a query.
3. Click the kebab context menu  at the top-right of the SQL editor and click **Restore**.

Set query description and view query info

To set a query description:

1. Click the  kebab context menu next to the query and click **Edit query info**.



2. In the **Description** text box, enter your description. Then, click **Save**. You can also view the history of the query, including when it was created and updated, in this dialog.

Favorite and tag queries

You can use favorites and tags to filter the lists of queries and dashboards displayed on your workspace landing page, and on each of the listing pages for dashboards and queries.

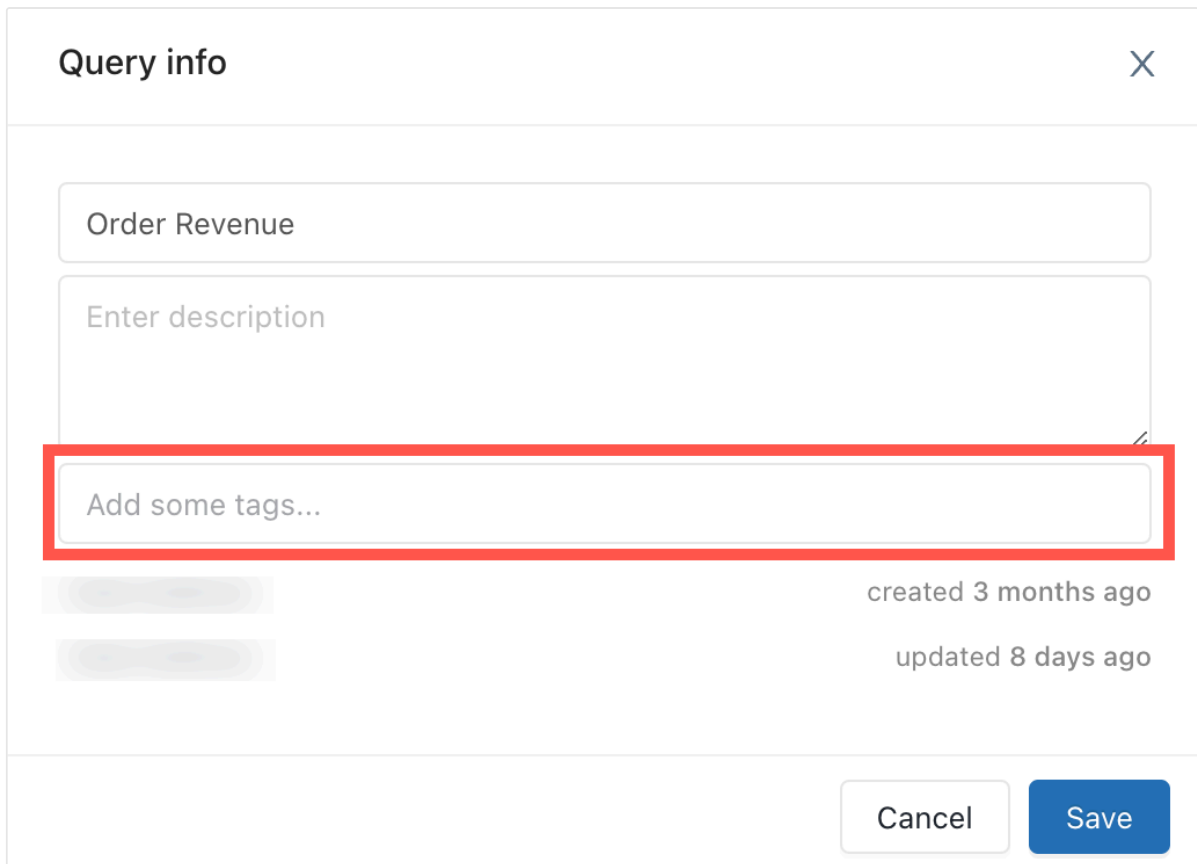
Favorites: To favorite a query, click the star to the left of its title in the Queries list. The star will turn yellow.

Tags: You can tag queries and dashboards with any meaningful string to your organization.

Add a tag

Add tags in the query editor.

1. Click the  kebab context menu next to the query and click **Edit query info**. A **Query info** dialog appears.
2. If the query has no tags applied, **Add some tags** shows in the text box where tags will appear. To create a new tag, type it into the box. To enter multiple tags, press  tab between entries.

A dialog box titled "Query info" with a close button (X) in the top right corner. It contains three input fields: "Order Revenue", "Enter description", and "Add some tags...". The "Add some tags..." field is highlighted with a red border. Below the input fields, there are two tags represented by rounded rectangles. The first tag has a close button (X) and the text "created 3 months ago". The second tag has a close button (X) and the text "updated 8 days ago". At the bottom right, there are two buttons: "Cancel" and "Save".

Query info

Order Revenue

Enter description

Add some tags...

created 3 months ago

updated 8 days ago

Cancel Save

3. Click **Save** to apply the tags and close the dialog.

Remove tags

1. Click the  kebab context menu next to the query and click **Edit query info**.
2. Click **X** on any tag you want to remove.
3. Click **Save** to close the dialog.

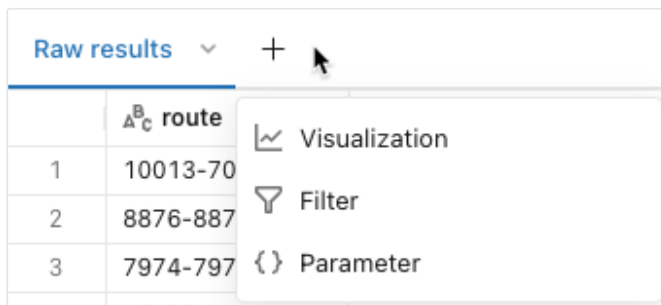
View query results

After a query runs, the results appear in the pane below it. The **New result table** is **ON** for new queries. If necessary, click the drop-down to turn it off. The images in this section use the new result table.

You can interact with and explore your query results using the result pane. The result pane includes the following features for exploring results.

Visualizations, filters, and parameters

Click the  to add a visualization, filter, or parameter. The following options appear:



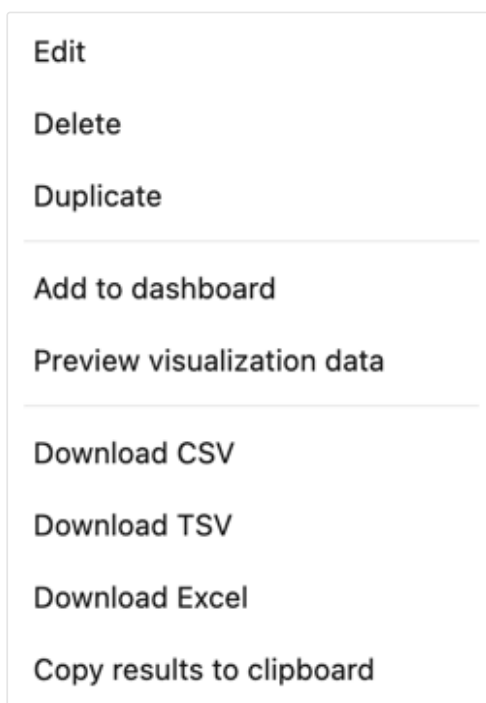
Visualization: Visualizations can help explore the result set. See [Notebook and SQL editor visualization types](#) for a complete list of available visualization types.

Filter: Filters allow you to limit the result set after a query has run. You can apply filters to selectively show different subsets of the data. See [Query filters](#) to learn how to use filters.

Parameter: Parameters allow you to limit the result set by substituting values into a query at runtime. See [Use named parameter markers](#) to learn how to apply parameters.

Edit, download, or add to a dashboard

Click the  in a results tab to view more options.



1. Click **Edit** to customize the results shown in the visualization.
2. Click **Delete** to delete the results tab.
3. Click **Duplicate** to clone the results tab.
4. Click **Add to dashboard** to add the visualization to an AI/BI dashboard. You are prompted to choose one of the following:
 - **Create new dashboard:** Enter a name for the new dashboard.
 - **Add to existing dashboard:** Search for an existing dashboard in the workspace.

In both cases, select the visualizations to include and optionally check **Automatically converts legacy parameter syntax to named parameter syntax, when possible** to convert legacy parameter syntax when adding to the dashboard. See [Dashboards](#) to learn how to edit your dashboard.

5. Click **Preview visualization data** to view the underlying data for the visualization.
6. Click any of the download options to download results. See the following description for details and limits.

Download results: You can download results as a CSV, TSV, or Excel file.

You can download up to approximately 1GB of results data from Databricks SQL in CSV and TSV format and up to 100,000 rows to an Excel file.

The final file download size might be slightly more or less than 1GB, as the 1GB limit is applied to an earlier step than the final file download.

NOTE

If you cannot download a query, your workspace administrator has disabled download for your workspace.

Past executions

You can view previous runs for the query, including the complete query syntax. Past executions open in read-only mode and include buttons to **Clone to new query** or **Resume editing**. This tab does not show [scheduled runs](#).



Past executions

Show past executions for:

Current query

Current draft

✓

```
select * from
samples.nyctaxi.trips
where trip_distance > 1 ...
```

rows, 5 ms, just now

✓

```
select * from
samples.nyctaxi.trips
limit 500
```

500 rows, 1.86 s, a minute ago

✓

```
select * from
samples.nyctaxi.trips
LIMIT 1000
```

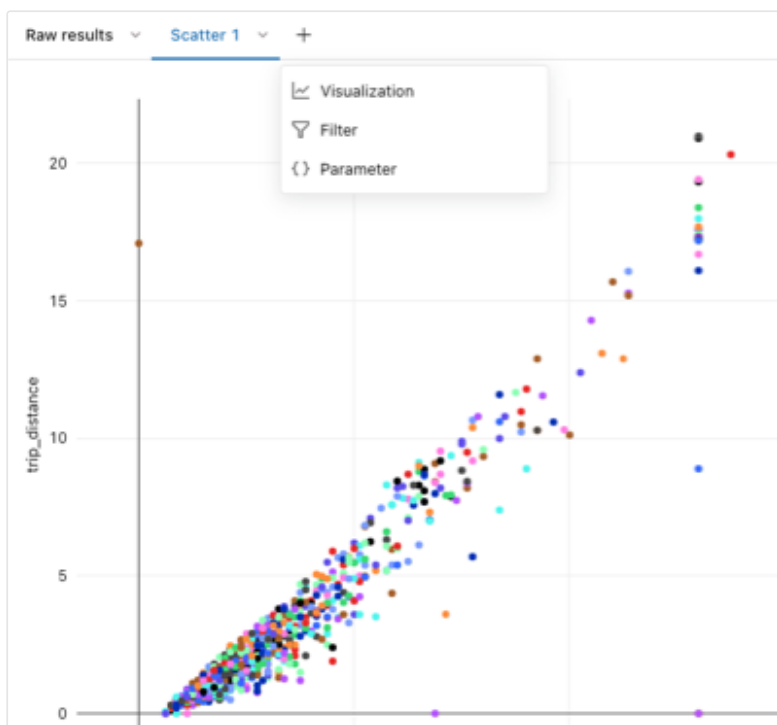
1000 rows, 1.71 s, an hour ago

Explore results

Returned query results appear below the query. The **Raw results** tab populates with the returned rows. You can use built-in filters to reorder the results by ascending or descending values. You can also use the filter to search for result rows that include a specific value.

	tpep_pickup_datetime	tpep_dropoff_datetime	trip_distance	fare_amount	pickup_zip	dropoff_zip
1	2016-02-01T03:06:48.000+00:00	2016-02-01T03:12:09.000+00:00	1.7	7	10003	10002
2	2016-02-01T07:50:59.000+00:00	2016-02-01T07:55:03.000+00:00	0.56	4.5	10110	10110
3	2016-02-01T08:10:34.000+00:00	2016-02-01T08:14:36.000+00:00	1.1	5.5	10001	10011
4	2016-02-01T08:57:50.000+00:00	2016-02-01T09:05:34.000+00:00	0.9	6.5	10153	10021
5	2016-02-01T09:15:43.000+00:00	2016-02-01T09:20:49.000+00:00	0.55	5	10022	10153
6	2016-02-01T09:22:46.000+00:00	2016-02-01T09:24:26.000+00:00	0.3	3.5	10167	10022
7	2016-02-01T10:04:20.000+00:00	2016-02-01T10:17:12.000+00:00	1.5	10	10103	10199
8	2016-02-01T11:31:26.000+00:00	2016-02-01T11:42:33.000+00:00	1.6	9	10017	10199
9	2016-02-01T11:31:09.000+00:00	2016-02-01T11:47:28.000+00:00	5.15	18	10162	10032
10	2016-02-01T12:19:05.000+00:00	2016-02-01T12:34:19.000+00:00	3.92	14.5	11370	11106
11	2016-02-01T14:19:32.000+00:00	2016-02-01T14:29:56.000+00:00	3.8	13	10005	10016
12	2016-02-01T15:04:42.000+00:00	2016-02-01T15:13:08.000+00:00	1.05	7	10119	10110
13	2016-02-01T15:20:35.000+00:00	2016-02-01T15:30:45.000+00:00	1.2	8.5	10019	10171
14	2016-02-01T15:39:23.000+00:00	2016-02-01T15:46:28.000+00:00	1.1	6.5	10026	10035
15	2016-02-01T16:06:45.000+00:00	2016-02-01T16:15:31.000+00:00	1.18	7.5	10023	10021
16	2016-02-01T16:44:06.000+00:00	2016-02-01T17:07:44.000+00:00	3.49	17.5	10019	10009
17	2016-02-01T17:04:34.000+00:00	2016-02-01T17:09:40.000+00:00	0.63	5	10001	10001
18	2016-02-01T17:45:57.000+00:00	2016-02-01T18:03:33.000+00:00	6.2	20	10029	11369
19	2016-02-01T18:19:38.000+00:00	2016-02-01T18:27:40.000+00:00	1.2	7.5	10031	10030
20	2016-02-01T18:45:48.000+00:00	2016-02-01T19:02:25.000+00:00	2.88	13.5	10021	10018
21	2016-02-01T19:34:27.000+00:00	2016-02-01T19:37:41.000+00:00	0.8	4.5	10110	10153
22	2016-02-01T20:30:31.000+00:00	2016-02-01T20:42:59.000+00:00	2.33	10.5	10065	10009
23	2016-02-01T20:39:52.000+00:00	2016-02-01T20:52:12.000+00:00	3.35	13	10282	10001
24	2016-02-01T20:49:59.000+00:00	2016-02-01T20:55:07.000+00:00	0.7	5.5	10019	10153

You can use tabs in the result pane to add visualizations, filters, and parameters.



Filter the list of saved queries in the queries window

In the queries window, you can filter the list of all queries by the list of queries you have created (**My Queries**), by favorites, and by tags.

Automate updates

You can use the **Schedule** button to set an automatic cadence for query runs. Automatic updates can help keep your dashboards and reports up-to-date with the most current data. Scheduled queries can also enable Databricks SQL alerts, a special type of scheduled task that sends notifications when a value reaches a specified threshold.

See [Schedule a query](#).

See [Databricks SQL alerts](#).

Share queries

The **Share** button lets you share your query with other users in your workspace. When sharing, choose the between the following options:

- **Run as owner (owner's credentials):** This setting means that viewers are able to see the same query results as the query owner. This applies to scheduled or manual query runs.
- **Run as viewer (viewers credentials):** This setting limits results to the viewer's assigned permissions.

See [Configure query permissions](#).

Next step

See [Access and manage saved queries](#) to learn how to work with queries with the Databricks UI.

Is this page
as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **Apr 29, 2026**

Mustache parameter syntax

❗ IMPORTANT

Mustache parameter syntax is supported in the legacy SQL editor only. Databricks recommends using [named parameter markers](#) for new queries. If you copy a query using mustache syntax into a notebook, AI/BI dashboard dataset editor, or Genie space, you must convert it to named parameter markers before it will run.

In the legacy SQL editor, any string wrapped in double curly braces (`{{ }}`) is treated as a query parameter. A widget appears above the results pane where you set the parameter value.

Add a mustache parameter

1. Press `Cmd + I`. The parameter is inserted at the cursor position and the **Add Parameter** dialog appears.
2. Enter the **Keyword**, optionally change the **Title**, and select a **Type**.
3. Click **Add Parameter**.
4. Set the parameter value in the widget.
5. Click **Apply Changes**.
6. Click **Save**.

Alternatively, type `{{ }}` directly in the query and click the gear icon in the parameter widget to configure it.

To re-run the query with a different value, update the widget and click **Apply Changes**.

Edit, remove, and reorder parameters

Edit: Click the gear icon beside the parameter widget. To prevent users who don't own the query from changing the parameter value, click **Show Results Only**.

Remove: Delete the parameter marker from the query. The widget is removed automatically.

Reorder: Click and drag parameter widgets to the desired order.

Widget types

The following widget types are available:

Text

Takes a string as input. Backslash, single, and double quotation marks are escaped automatically. Databricks adds quotation marks around the value.

SQL

```
SELECT * FROM samples.tpch.customer WHERE c_name = {{ name_param }}
```

Number

Takes a number as input.

SQL

```
SELECT * FROM users WHERE age = {{ number_param }}
```

Date and Time

Parameterizes date and timestamp values. Three options are available:

 Ask Genie

Type	Precision
Date	Day
Date and Time	Minute
Date and Time (with seconds)	Second

All date and time values are passed as string literals and must be wrapped in single quotation marks in your query:

SQL

```
SELECT * FROM usage_logs WHERE date = '{{ date_param }}'
```

When you select a **Range** option, Databricks creates two parameters using `.start` and `.end` suffixes:

SQL

```
SELECT * FROM usage_logs
WHERE modified_time > '{{ date_range.start }}'
AND modified_time < '{{ date_range.end }}'
```

NOTE

The Date Range widget returns correct results only for `DATE`-type columns. For timestamp columns, use a Date and Time Range widget.

Dynamic date values: Date widgets include a blue lightning bolt icon. Click it to select dynamic values such as `today`, `yesterday`, `this week`, `last week`, `last month`, or `last year`. These values update automatically.

IMPORTANT

Dynamic date values are not compatible with scheduled queries.

Dropdown List

Restricts input to a predefined static list. Supports both single-value and multi-value selection.

- **Single value:** Wrap the parameter in single quotation marks in the query.
- **Multi-value:** Enable **Allow multiple values** in the widget settings. Use the **Quotation** option to control whether values are wrapped in single quotes, double quotes, or none. Update your `WHERE` clause to use `IN`:

SQL

```
SELECT * FROM orders WHERE status IN ( {{ status_param }} )
```

With double quotation marks selected, the query resolves to: `WHERE status IN ("value1", "value2", "value3")`.

Dropdown List values are strings. To use dates or timestamps, enter them in the format your data source requires.

Query-Based Dropdown List

Populates dropdown choices from a saved query. Behaves like Dropdown List but the options are dynamic.

1. Under **Type** in the settings panel, select **Query Based Dropdown list**.
2. Click the **Query** field and select a saved query.

If the query returns more than one column, Databricks uses the first column. If the query returns columns named `name` and `value`, the widget displays the `name` column but passes the associated `value` to the query at runtime.

Example:

SQL

```
SELECT user_uuid AS 'value', username AS 'name' FROM users
```

 Ask Genie

value	name
1001	John Smith
1002	Jane Doe
1003	Bobby Tables

The value passed to the database at runtime is `1001`, `1002`, or `1003`, not the display name.

NOTE

Performance degrades if the source query returns a large number of records.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Work with parameter widgets

When you add a [named parameter marker](#) to a query, Databricks displays a parameter widget in the UI. Widgets let users set parameter values without editing the query directly. You can configure each widget's type, title, and default value.

Parameter widgets are supported in the SQL editor, notebooks, AI/BI dashboards, and Genie Spaces, but behave differently across these surfaces. This page describes parameter widgets in the SQL editor. For other surfaces, see:

- [Work with dashboard parameters](#) for dashboards
- [Add query parameters](#) for Genie Spaces
- [Databricks widgets](#) for notebooks

In the SQL editor, any parameter type (String, Integer, Decimal, Date, Timestamp) can use any widget type.

Configure a parameter widget

1. Add a named parameter marker to your query. A widget appears in the UI.

2. Click the gear icon beside the widget to open the widget dialog.

Parameter name

Widget label

Widget type

Text ▾

Parameter type ⓘ

String ▾

Default parameter value

Remove parameter

3. Set the following fields:

- **Parameter name:** The parameter name as it appears in the query. If you change the parameter name, in the widget dialog, you must also change it in the query.
- **Widget label:** A string that described the widget
- **Widget type:** Controls how users input the value. See [Widget types](#) below.
- **Parameter type:** The parameter data type. See [Parameter types](#).

4. Click away from the widget dialog to save your changes.

Edit, remove, and reorder widgets

Edit: Click the gear icon beside the widget to reopen the settings pane.

Remove: Delete the parameter marker from the query. The widget is removed automatically.

Reorder: Use the drag handle to the left of a widget to reorder.

Widget types

Databricks supports the following widget types for query parameters:

Widget type	Description
Dropdown	Users must choose from a predefined list.
Combobox	Users can choose from a predefined list or type a custom value.
Text input	Accepts any free-form value with no suggestions.
Multiselect	Users can select more than one value from a predefined list.
Dynamic dropdown	Populates choices from a saved query instead of a static list.
Date and Timestamp range	Defines a start and end range using <code>.min</code> and <code>.max</code> parameters.

Text input

Accepts a free-form value directly from the user. Use this widget when no predefined options are needed.

SQL

```
SELECT * FROM samples.tpch.region WHERE r_name = :region_param
```

Dropdown

Presents a predefined list of values. Users must select from the list — free-form entry is not allowed. Enter allowed values in the settings pane, one per line.

SQL

```
SELECT * FROM samples.tpch.orders WHERE o_orderstatus = :status_param
```

To create a dropdown widget type:

1. Click the gear icon beside the `:status_param` widget.
2. Set **Widget type** to **Dropdown**.
3. Set **Parameter type** to **String**.
4. Enter values into the **Choices for parameter value** text entry field. Click **Add** or press enter between each value.

Combobox

Presents a predefined list of suggested values but also allows users to type a custom value not in the list. Use a combobox when common options are useful for convenience but you want to allow free-form input.

SQL

```
SELECT * FROM samples.tpch.part WHERE p_brand = :brand_param
```

Multiselect

Allows users to select more than one value from a predefined list. The selected values are passed to the query as a collection.

SQL

```
SELECT * FROM samples.nyctaxi.trips WHERE
  array_contains(
    TRANSFORM(SPLIT(:list_parameter, ','), s -> TRIM(s)),
    CAST(dropoff_zip AS STRING)
  )
```

To add options for a multiselect dropdown list:

1. Click the gear icon beside the `list_parameter` widget.
2. Set **Widget type** to **Multiselect**.
3. Set **Parameter type** to **String**.
4. Enter values into the **Choices for parameter value** text entry field. Click **Add** or press enter between each value.

Dynamic dropdown

NOTE

Dynamic dropdown widgets are available in the SQL editor only, not in notebooks.

Populates the list of choices from a saved query instead of a static list. As the underlying data changes, the available options update automatically.

To use a dynamic dropdown:

1. Create and save a query that returns the values you want in the dropdown:

SQL

```
SELECT DISTINCT c_mktsegment FROM samples.tpch.customer ORDER BY
c_mktsegment
```

2. In a new or existing query, add a named parameter marker:

SQL

 Ask Genie

```
SELECT c_custkey, c_name, c_acctbal
FROM samples.tpch.customer
WHERE c_mktsegment = :segment_param
```

3. Click the gear icon beside the `segment_param` widget.
4. Set **Widget type** to **Dynamic dropdown**.
5. Click the **Query** field to open the **Select existing query** dialog. Select the saved query from step 1, then click **Select**.
6. Choose a **Default parameter value**.
7. Click **Apply Changes**.

Date and Timestamp range

Date and Timestamp parameters support a **Range** widget type. When selected, Databricks creates two parameters using `.min` and `.max` suffixes to define the start and end of the range.

SQL

```
SELECT * FROM samples.nyctaxi.trips
WHERE tpep_pickup_datetime
BETWEEN CAST(:date_range_min AS TIMESTAMP) AND CAST(:date_range_max AS
TIMESTAMP)
```

Click the blue lightning bolt icon to select dynamic values such as `today`, `yesterday`, `this week`, `last week`, `last month`, or `last year`. These values update automatically.

ⓘ IMPORTANT

Dynamic date values are not compatible with scheduled queries.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback